

# 15 – Cookies & sessions

A way to memorize information on the client or on the server

Jérôme Landré – [jerome.landre@ju.se](mailto:jerome.landre@ju.se)

# OUTLINE

- 1) HTTP is stateless
- 2) Cookies
- 3) Cookies and Express
- 4) Sessions
- 5) Sessions and Express

# 1) HTTP IS STATELESS

- Remember: HTTP is a **stateless** protocol: no memorization in between two request/response => you cannot save information in memory



# 1) HTTP IS STATELESS

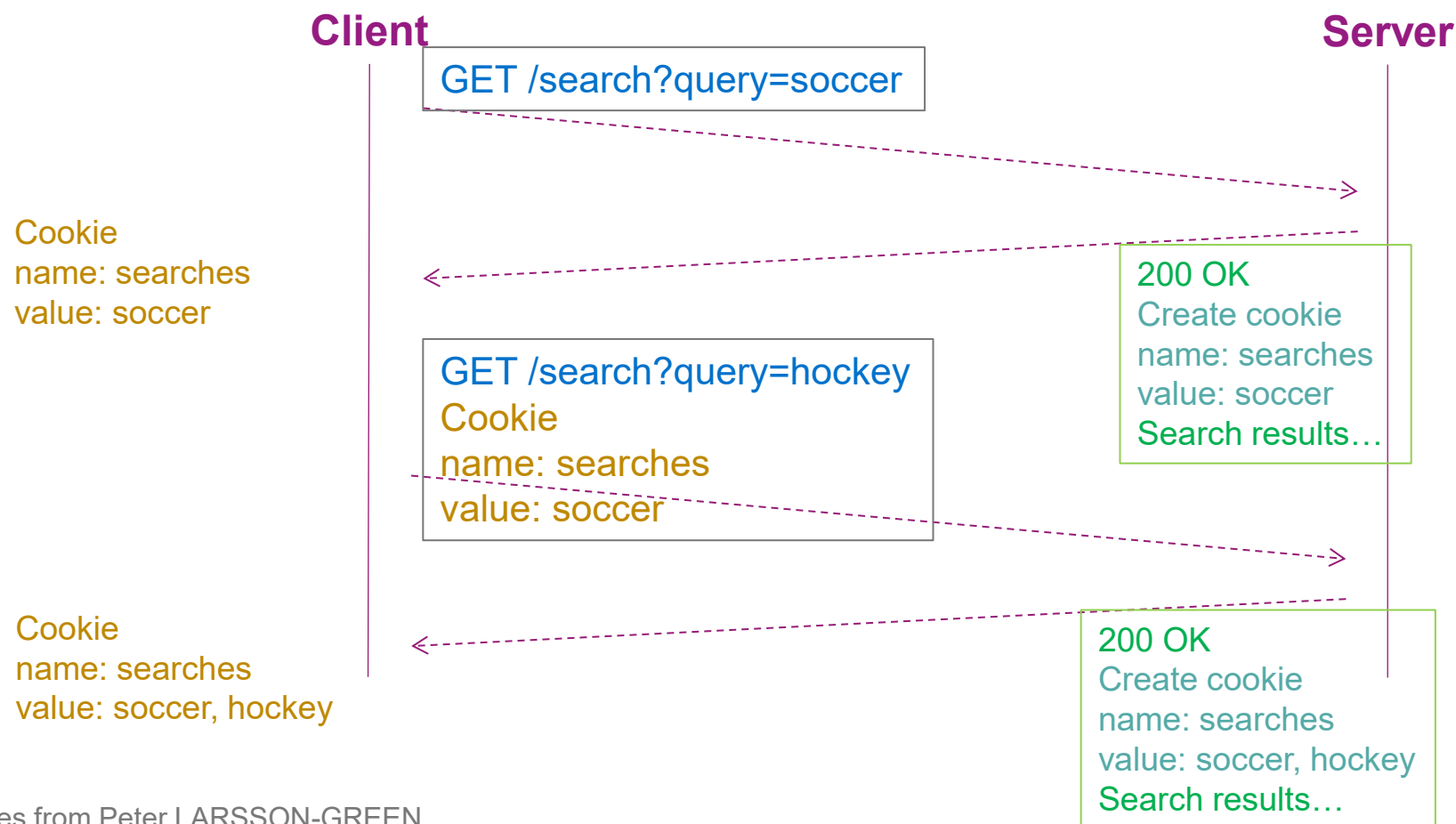
- Solution:
  - Store information on the **client**: **Cookies**
  - Store information on the **server**: **Sessions**
- Easy to memorize:
  - **C**lient: **C**ookies
  - **S**erver: **S**essions

## 2) COOKIES

- Cookies are text information stored in a file on the client
- When using Cookies?
  - When we need some memorization of information: to remember the client's state between different requests:
    - In a web shop, which items has the client in the basket?
    - On a search engine, what are the previous words the user has searched for?
    - On a news site, which type of news are you interested in?

## 2) COOKIES

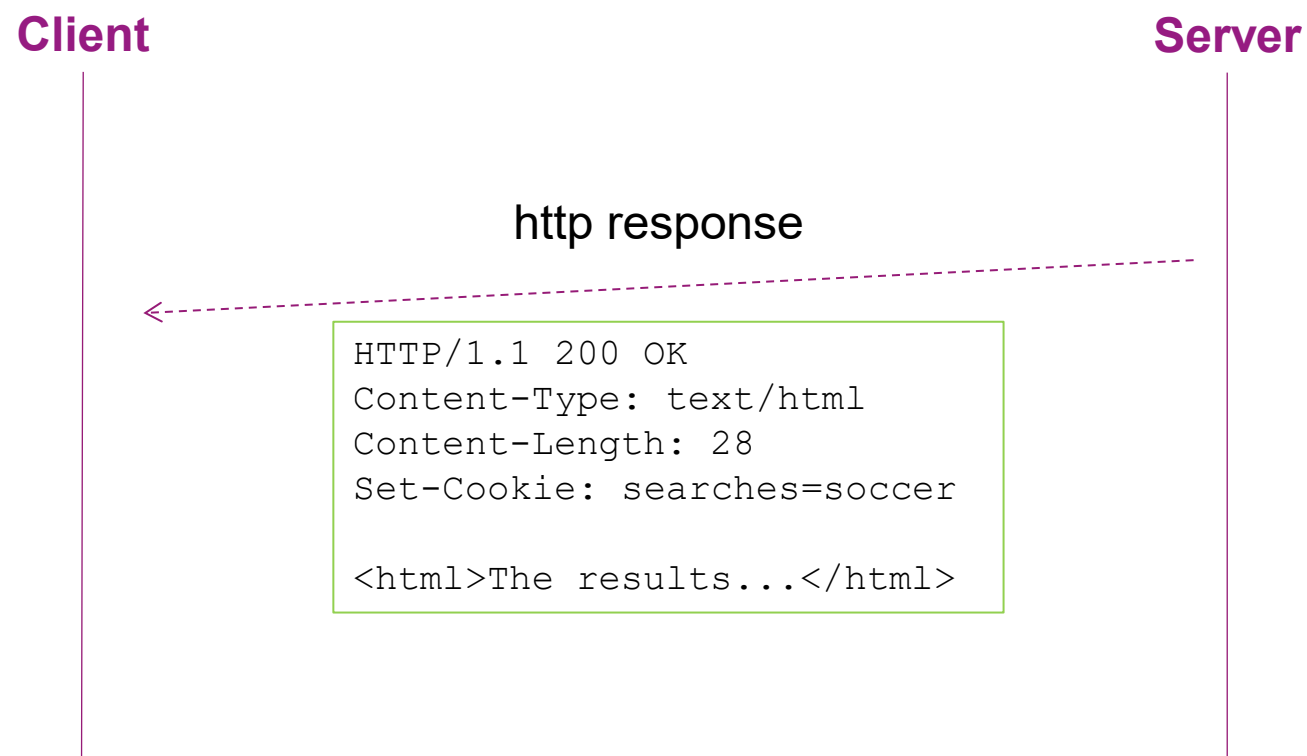
- Cookies are text information stored in a file on the client:



Examples from Peter LARSSON-GREEN

## 2) COOKIES

- Cookies are text information stored in a file on the client:



## 2) COOKIES

- Cookies are text information stored in a file on the client:





## 2) COOKIES

- Cookies are described in the HTTP State Management Mechanism RFC:  
<https://tools.ietf.org/html/rfc6265>
- Be careful when using cookies, they are regulated by EU laws:
  - The ePrivacy Directive ("The Cookie Law"):  
<https://gdpr.eu/cookies/>
  - General Data Protection Regulation (GDPR):  
<https://www.cookiebot.com/en/gdpr-cookies/>

### 3) COOKIES AND EXPRESS

- To create a cookie in Express:

```
response.cookie("NameOfTheCookie", "ValueOfTheCookie")  
response.cookie("firstName", "Jerome")
```

- To read a cookie:

```
// reads a cookie  
const cookieParser = require('cookie-parser') // put this line on top of the file...  
// ...with all the other require lines
```

```
app.use(cookieParser()) // put this line at the beginning of your file...  
// ...after the line const app = express()
```

```
const lastVisit = parseInt(request.cookies.lastVisit)  
let firstName = request.cookie.firstName
```

```
npm install cookie-parser
```

**Remark:  
blank page  
returned!**

```
// -----  
// COOKIES  
// -----  
// creates a cookie  
app.get("/create-cookie", function(request, response){  
  console.log("Route: "+request.url)  
  response.cookie("lastVisit", Date.now())  
  response.end()  
})  
  
// reads a cookie  
const cookieParser = require('cookie-parser') // put this line on top of the file...  
// ...with all the other require lines  
  
app.use(cookieParser()) // put this line at the beginning of your file...  
// ...after the line const app = express()  
  
app.get("/log-cookie", function(request, response){  
  let counter = 1  
  if(request.cookies.counter){  
    counter = parseInt(request.cookies.counter) + 1  
  }  
  response.cookie("counter", counter)  
  
  console.log("Route: "+request.url)  
  console.log("Cookies: "+JSON.stringify(request.cookies)) // transforms an object into a string...  
  //... good for debugging!  
  
  const lastVisit = parseInt(request.cookies.lastVisit)  
  console.log("Your last visit: "+lastVisit)  
  const counterCookie = parseInt(request.cookies.counter)  
  console.log("Number of visits: "+counterCookie)  
  response.end()  
})
```

```
PS C:\Users\lanjer\Desktop\web-dev-fun\portfolio-jl> node app.js
Server running and listening on port 8080...
Route: /log-cookie
Cookies: {}
Your last visit: NaN
Number of visits: NaN
Route: /create-cookie
Route: /log-cookie
Cookies: {"counter":"1","lastVisit":"1695711490703"}
Your last visit: 1695711490703
Number of visits: 1
Route: /log-cookie
Cookies: {"lastVisit":"1695711490703","counter":"2"}
Your last visit: 1695711490703
Number of visits: 2
Route: /create-cookie
Route: /log-cookie
Cookies: {"counter":"3","lastVisit":"1695711508725"}
Your last visit: 1695711508725
Number of visits: 3
Route: /log-cookie
Cookies: {"lastVisit":"1695711508725","counter":"4"}
Your last visit: 1695711508725
Number of visits: 4
```

⌕ Headers Preview Response Initiator Timing Cookies

▼ General

Request URL: http://localhost:8080/log-cookie  
Request Method: GET  
Status Code: 200 OK  
Remote Address: [::1]:8080  
Referrer Policy: strict-origin-when-cross-origin

▼ Response Headers ☒ Raw

HTTP/1.1 200 OK  
X-Powered-By: Express  
Set-Cookie: counter=5; Path=/  
Date: Tue, 26 Sep 2023 06:58:54 GMT  
Connection: keep-alive  
Keep-Alive: timeout=5  
Content-Length: 0

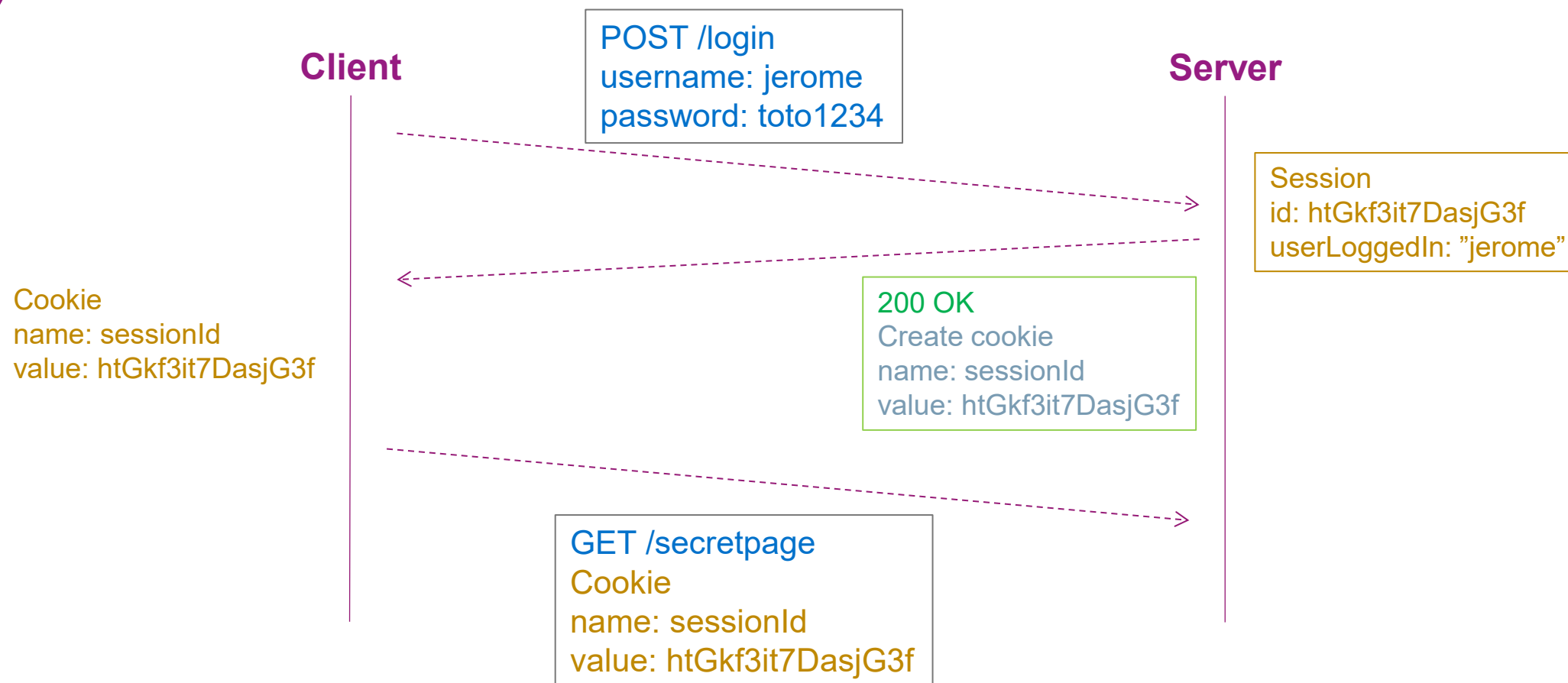
▼ Request Headers ☒ Raw

GET /log-cookie HTTP/1.1  
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8  
Accept-Encoding: gzip, deflate, br  
Accept-Language: en-US,en  
Cache-Control: max-age=0  
Connection: keep-alive  
Cookie: lastVisit=1695711508725; counter=4  
Host: localhost:8080  
Sec-Fetch-Dest: document  
Sec-Fetch-Mode: navigate  
Sec-Fetch-Site: none  
Sec-Fetch-User: ?1  
Sec-GPC: 1  
Upgrade-Insecure-Requests: 1  
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36  
sec-ch-ua: "Brave";v="117", "Not;A=Brand";v="8", "Chromium";v="117"  
sec-ch-ua-mobile: ?0  
sec-ch-ua-platform: "Windows"

### 3) COOKIES AND EXPRESS

- **HOWEVER** because Cookies are text information stored in a file on the client
  - The client can **read** the values: **BAD** for security reasons!
  - The client can **change** the values of the cookies: **EVEN WORST** for security!
- => For **sensitive information**, we **MUST** use sessions (server-side stored information)

## 4) SESSIONS • Sessions are text information stored in a file on the server:



## 4) SESSIONS

- In this case, if the client changes the sessionId (it is stored in a file on the client), the client won't be able to connect to the “/secretpage” route that needs the session information
- Sessions are more **SECURE** than cookies because there are stored on the server, the client cannot change them!



## 4) SESSIONS AND EXPRESS

- To create a session in Express:

```
const express = require('express')
```

```
const session = require('express-session')
```

```
npm install express-session
```

```
const app = express()
```

```
// MIDDLEWARE
```

```
app.use(session({
```

```
  "saveUninitialized": false,
```

```
  "resave": false,
```

```
  "secret": "This123IsASecret678Sentence"
```

```
}))
```

## 4) SESSIONS AND EXPRESS

```
// creates a session for page counting
app.get("/create-session", function(request, response){
  console.log("Route: "+request.url)
  let counter = 1
  if(request.session.counter){
    counter = request.session.counter + 1
  }
  request.session.counter = counter
  request.session.firstName = "Jerome"
  response.end()
})
```

## 4) SESSIONS AND EXPRESS

```
app.get("/log-session", function(request, response){  
  console.log("Route: "+request.url)  
  console.log("Session: "+JSON.stringify(request.session)) // transforms an object into a string...  
  //... good for debugging!  
  const counter = parseInt(request.session.counter)  
  console.log("Number of visits: "+counter)  
  const fName = request.session.firstName  
  console.log("Hello, "+fName)  
  response.end()  
})
```

**Remark:  
blank page  
returned!**

```
// -----  
// SESSIONS  
// -----  
  
const session = require('express-session') // put this line on top of the file...  
// ...with all the other require lines  
  
// put these line at the beginning of your file...  
// ...after the line const app = express()  
app.use(session({  
  saveUninitialized: false,  
  resave: false,  
  secret: 'This123IsASecret678Sentence'  
}))  
  
// creates a session for page counting  
app.get("/create-session", function(request, response){  
  console.log("Route: "+request.url)  
  let counter = 1  
  if(request.session.counter){  
    counter = request.session.counter + 1  
  }  
  request.session.counter = counter  
  request.session.firstName = "Jerome"  
  response.end()  
})  
  
app.get("/log-session", function(request, response){  
  console.log("Route: "+request.url)  
  console.log("Session: "+JSON.stringify(request.session)) // transforms an object into a string...  
  //... good for debugging!  
  
  const counter = parseInt(request.session.counter)  
  console.log("Number of visits: "+counter)  
  const fName = request.session.firstName  
  console.log("Hello, "+fName)  
  response.end()  
})
```

```
PS C:\Users\lanjer\Desktop\web-dev-fun\portfolio-jl> node app.js
Server running and listening on port 8080...
Route: /log-session
Session: {"cookie":{"originalMaxAge":null,"expires":null,"httpOnly":true,"path":"/"}}
Number of visits: NaN
Hello, undefined
Route: /create-session
Route: /log-session
Session: {"cookie":{"originalMaxAge":null,"expires":null,"httpOnly":true,"path":"/"},"counter":1,"firstName":"Jerome"}
Number of visits: 1
Hello, Jerome
Route: /create-session
Route: /log-session
Session: {"cookie":{"originalMaxAge":null,"expires":null,"httpOnly":true,"path":"/"},"counter":2,"firstName":"Jerome"}
Number of visits: 2
Hello, Jerome
█
```

×

Headers

Preview

Response

Initiator

Timing

Cookies

▼ General

Request URL:

http://localhost:8080/log-session

Request Method:

GET

Status Code:

200 OK

Remote Address:

[::1]:8080

Referrer Policy:

strict-origin-when-cross-origin

▼ Response Headers

Raw

HTTP/1.1 200 OK

X-Powered-By: Express

Date: Tue, 26 Sep 2023 08:34:48 GMT

Connection: keep-alive

Keep-Alive: timeout=5

Transfer-Encoding: chunked

▼ Request Headers

Raw

GET /log-session HTTP/1.1

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8

Accept-Encoding: gzip, deflate, br

Accept-Language: en-US,en

Cache-Control: max-age=0

Connection: keep-alive

Cookie: connect.sid=s%3Ars\_suNH9wU4qEcA06\_knzZA9ZYGc74b3.MM7jeTdazo7lnU85wGNYa%2FrgyXNJXNHITeZi6ba9px8

Host: localhost:8080

Sec-Fetch-Dest: document

Sec-Fetch-Mode: navigate

Sec-Fetch-Site: none

Sec-Fetch-User: ?1

Sec-GPC: 1

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36

sec-ch-ua: "Brave";v="117", "Not;A=Brand";v="8", "Chromium";v="117"

sec-ch-ua-mobile: ?0

sec-ch-ua-platform: "Windows"

| Request Cookies                                            |                                                                |           |      |                 |      |          |        |          |               |          |  |
|------------------------------------------------------------|----------------------------------------------------------------|-----------|------|-----------------|------|----------|--------|----------|---------------|----------|--|
| <input type="checkbox"/> show filtered out request cookies |                                                                |           |      |                 |      |          |        |          |               |          |  |
| Name                                                       | Value                                                          | Domain    | Path | Expires / Ma... | Size | HttpOnly | Secure | SameSite | Partition Key | Priority |  |
| connect.sid                                                | s%3Ars_suNH9wU4qEcA06_knzZA9ZYGc74b3.MM7jeTdazo7lnU85wGNYa%... | localhost | /    | Session         | 93   | ✓        |        |          |               | Medium   |  |

# ANY QUESTIONS?